

oauth2-passkey

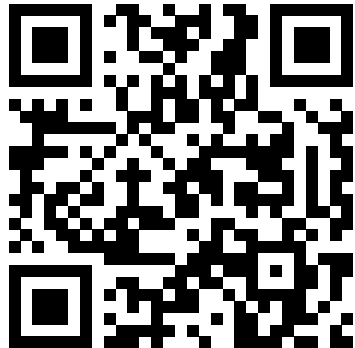
Passwordless Authentication Library for Rust
Kimitoshi Takahashi

Tokyo Rust | date TBD

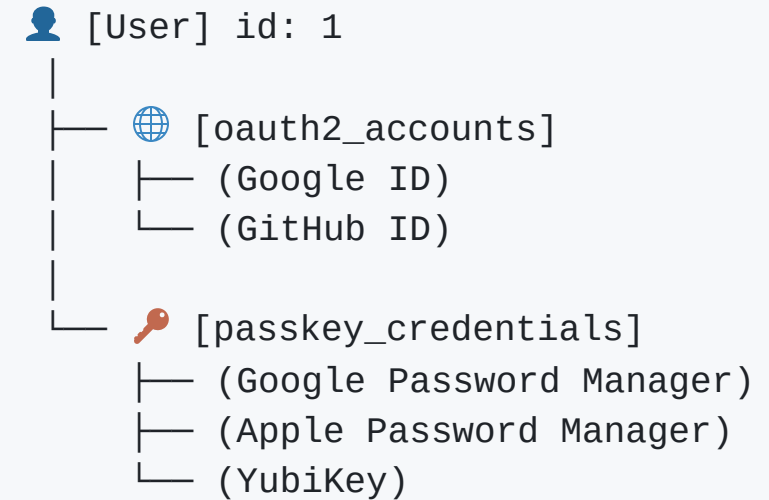
Live Demo

Demo

1. Google OAuth2/OIDC Login to create a user
2. Passkey Promotion -> register
3. Login with either Passkey or Google OAuth2
4. FedCM: browser-native account picker (if time allows)



Try it out & give me feedback!



What the Demo Just Showed

Step	What happened
Google OAuth2 login	New user created, no password ever set
Passkey Promotion	Prompted to add a Passkey <i>after</i> OAuth2 login
Re-login with Passkey	No redirect to Google — instant, local biometric check
Account page	One user, multiple linked credentials (OAuth2 + Passkey)
Everywhere	Session cookie + CSRF token, issued automatically

Two independent logins, one identity. That's the integration story.

Motivation

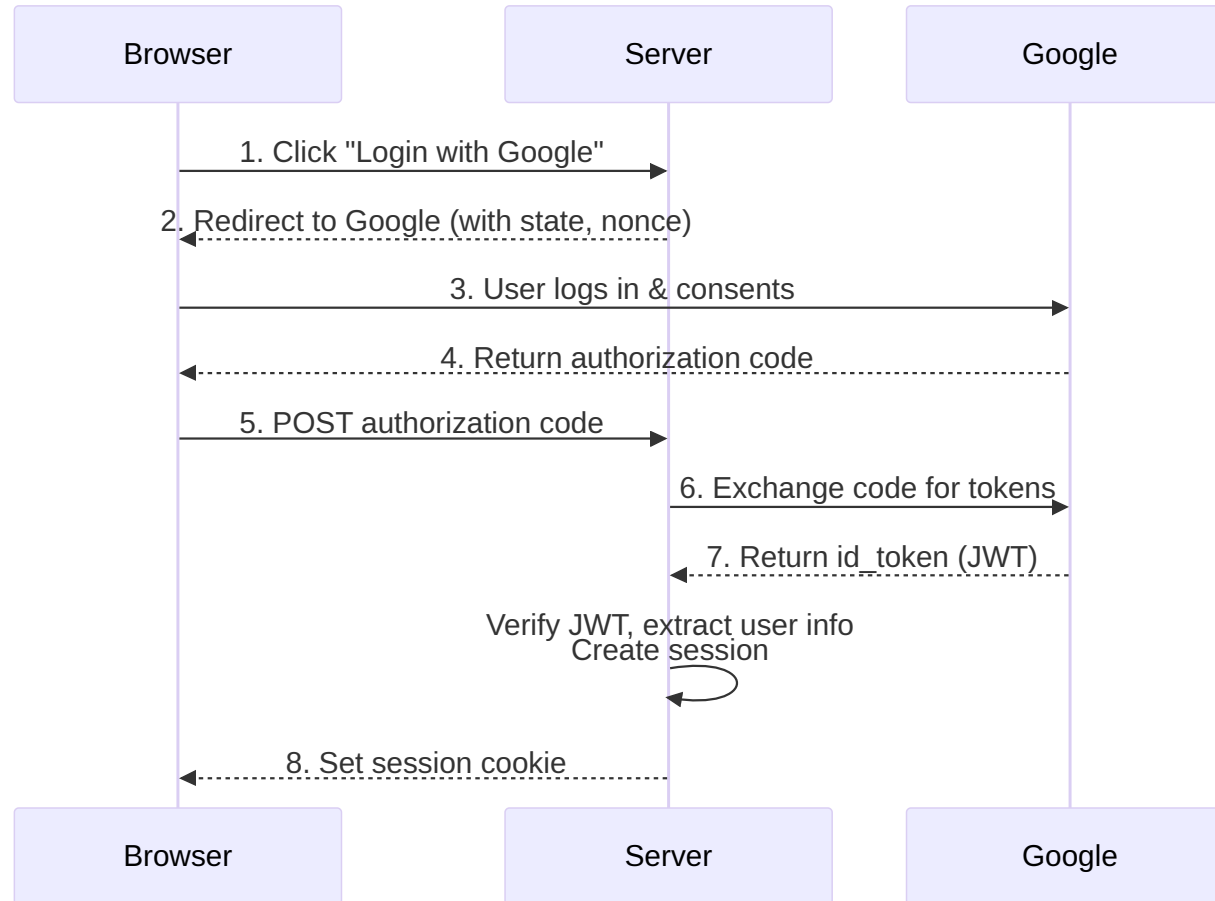
I wanted to build **exactly what you just saw** — in Rust.

- **OAuth2/OIDC**: delegate auth to trusted providers — no passwords to manage
- **Passkey**: phishing-resistant, biometrics/hardware key — no server-side secrets
- Existing Rust ecosystem: `webauthn-rs` (Passkey only), `oauth2` crate (OAuth2 only)
- Nothing combined both **and** managed sessions/account linking → built one → published to **crates.io**

Agenda

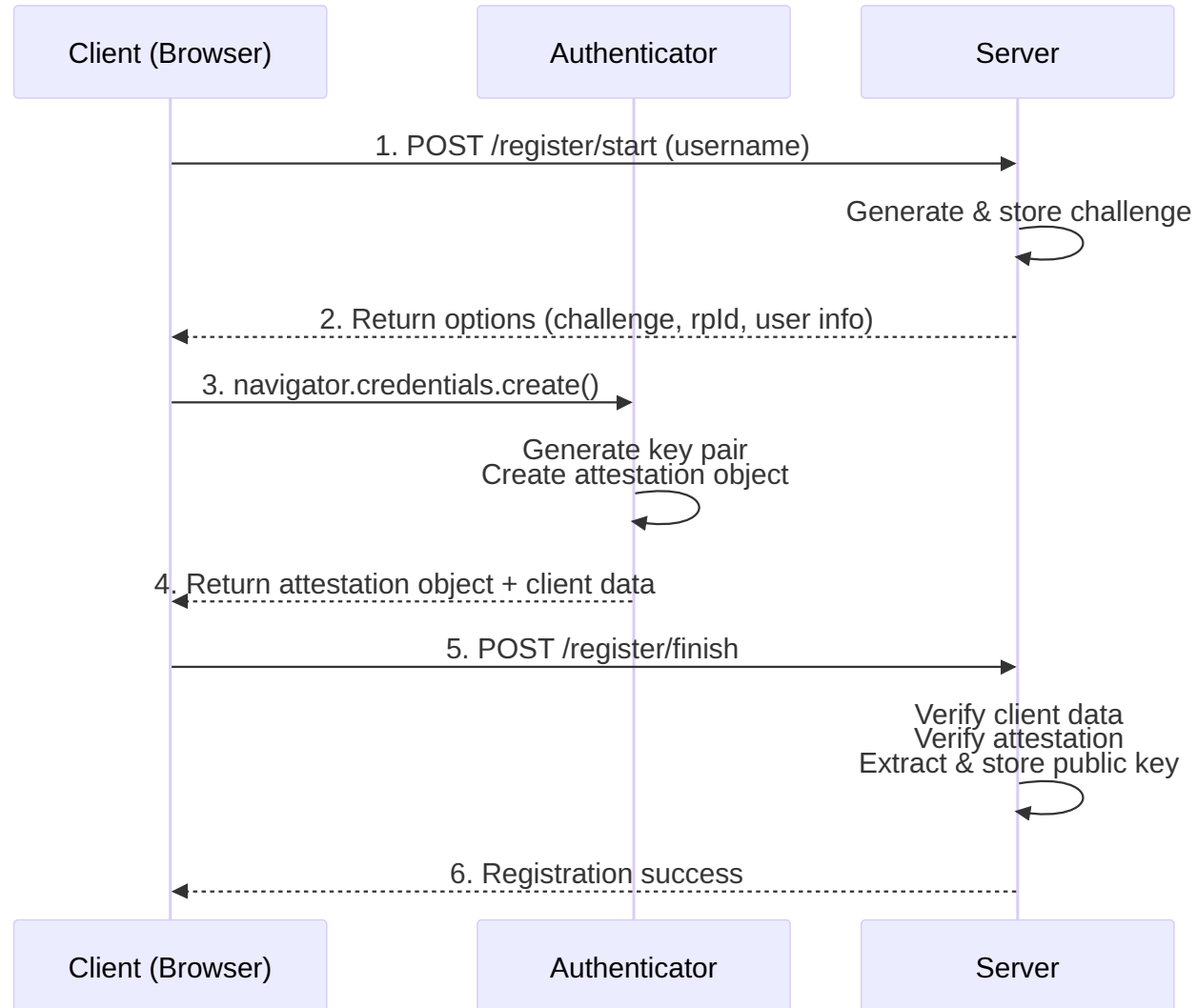
1. How OAuth2 & Passkey Work
2. Using the Library
3. Storage & the LazyLock Design Decision
4. Rust Design Patterns Under the Hood
5. Integrating with Your App
6. Live Coding: Putting It All Together
7. Wrap-up

How OAuth2/OIDC Works



*Page-redirect: Google → code → id_token
→ session*

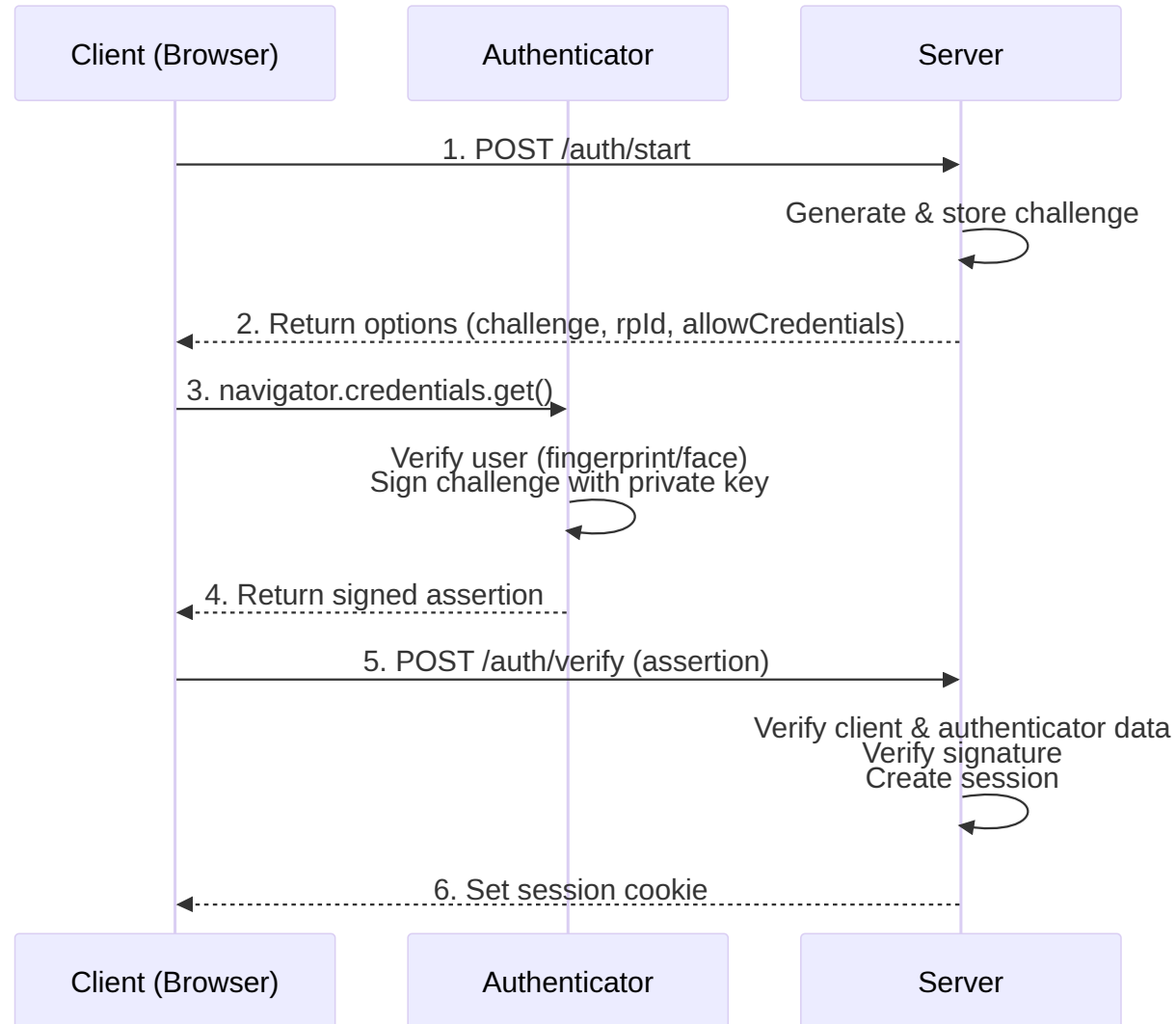
How Passkey/WebAuthn Registration Works



One-time setup: generate key pair → store public key on server

Private key stays on device, never leaves the authenticator

How Passkey/WebAuthn Authentication Works



JavaScript-driven: challenge → sign → verify → session

Authenticators: Google Password Manager, Apple, Windows Hello, YubiKey, ...

OAuth2 vs Passkey: Trust Model

OAuth2/OIDC

- Depends on an external provider (Google, GitHub, ...)
- Provider outage → you can't log in
- Provider decides what identity info you get

Passkey/WebAuthn

- Self-contained — no external service at login time
- Your server + the user's authenticator, nothing else
- You own the entire trust chain

Why offer both: OAuth2 for onboarding friction, Passkey for the fast/offline-friendly path afterward.

Using the Library

.env Setup (Minimal)

```
ORIGIN='http://localhost:3001'

# Google OAuth2 credentials
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
OAUTH2_GOOGLE_CLIENT_SECRET='xxx'

# SQLite + in-memory cache (no DB setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
GENERIC_CACHE_STORE_URL='memory'
```

- Data store: SQLite, PostgreSQL, MySQL
- Cache store: memory, Redis

How to Use

```
use oauth2_passkey_axum::{
    AuthUser, oauth2_passkey_full_router, // 1, Import
};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;          // 2. Initialize

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. Merge router

    // Auth endpoints are now at /o2p/*
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Under `/o2p/*`, we have:

- OAuth2 endpoints and Passkey endpoints
- Built-in login UI, account management, and admin panel

Page Protection: AuthUser Extractor

```
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse {
    match user {
        Some(u) => format!("Hello, {}!", u.label),
        None    => "Hello, anonymous!".to_string(),
    }
}
```

Implemented via Axum's `FromRequestParts` trait:

- `AuthUser` → GET: redirect to login, others: 401
- `Option<AuthUser>` → `OptionalFromRequestParts` impl returns `None` on failure — no redirect

Limitation: always hits DB, GET always redirects. Middleware gives more control: skip DB, or return 401 even on GET.

Page Protection: Middleware

Middleware gives more control: **skip DB, or return 401 even on GET.**

```
let app = Router::new()
  .route("/api/1", get(h1)
    .route_layer(from_fn(is_authenticated_401))) // <-- No DB query
  .route("/api/2", get(h2)
    .route_layer(from_fn(is_authenticated_user_401))); // <-- DB query, user info available
```

```
// In handler: access user or CSRF token via Extension
async fn h1(Extension(csrf): Extension<CsrfToken>) { ... }
async fn h2(Extension(user): Extension<AuthUser>) { ... }
```

Page Protection: Middleware Variants

Variant	Unauthenticated	DB Query	Handler Extension
is_authenticated_401	401	No	CsrfToken
is_authenticated_user_401	401	Yes	AuthUser
is_authenticated_redirect	Redirect to login	No	CsrfToken
is_authenticated_user_redirect	Redirect to login	Yes	AuthUser

Storage & the LazyLock Design Decision

Switch DB by Changing `.env`

```
# SQLite (dev/demo - no setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# PostgreSQL
GENERIC_DATA_STORE_TYPE=postgres
GENERIC_DATA_STORE_URL='postgres://user:pass@localhost:5432/mydb'

# MySQL / MariaDB
GENERIC_DATA_STORE_TYPE=mysql
GENERIC_DATA_STORE_URL='mysql://user:pass@localhost:3306/mydb'

# Cache: in-memory or Redis
GENERIC_CACHE_STORE_TYPE=memory          # or: redis
GENERIC_CACHE_STORE_URL='memory'         # or: redis://localhost:6379
```

No code changes. Just swap the env vars and restart.

How Multi-DB Support Works Internally

1. LazyLock reads env at startup:

```
static GENERIC_DATA_STORE: LazyLock<Mutex<Box<dyn DataStore>>>
= LazyLock::new(|| {
    let db_type = env::var("GENERIC_DATA_STORE_TYPE");
    let db_url = env::var("GENERIC_DATA_STORE_URL");

    match db_type {
        "sqlite" => Box::new(SqliteDataStore { pool }),
        "postgres" => Box::new(PostgresDataStore { pool }),
        "mysql" => Box::new(MySqlDataStore { pool }),
    }
});
```

2. Trait + impl per backend:

```
// DataStore: Send + Sync – safe to hold in a global LazyLock

impl DataStore for SqliteDataStore {
    fn as_sqlite(&self) -> Option<&Pool<Sqlite>> { Some(&self.pool) }
    fn as_postgres(&self) -> Option<&Pool<Postgres>> { None }
    fn as_mysql(&self) -> Option<&Pool<MySql>> { None }
}
```

3. Callers dispatch via trait:

```
let store = GENERIC_DATA_STORE.lock().await;
match (store.as_sqlite(), store.as_postgres(), store.as_mysql()) {
    (Some(pool), _, _) => get_all_users_sqlite(pool).await,
    (_, Some(pool), _) => get_all_users_postgres(pool).await,
    (_, _, Some(pool)) => get_all_users_mysql(pool).await,
    _ => Err(UserError::Storage("Unsupported db".into())),
}
```

Why Not Axum State? The State Threading Problem

With Axum State — state must flow through every layer:

```
// parent doesn't use DB, but must accept State
// just to pass it down to grandchild
async fn parent(
    State(state): State<AppState>,
) -> impl IntoResponse {
    child(state).await
}

async fn child(state: AppState) -> impl IntoResponse {
    grandchild(&state).await
}

// Only grandchild actually needs DB
async fn grandchild(state: &AppState) -> impl IntoResponse {
    let users = db::get_users(&state.auth_pool).await;
    // ...
}
```

With LazyLock — no threading needed:

```
// parent and child have no idea DB exists
async fn parent() -> impl IntoResponse {
    child().await
}

async fn child() -> impl IntoResponse {
    grandchild().await
}

// grandchild accesses DB directly
async fn grandchild() -> impl IntoResponse {
    let store = GENERIC_DATA_STORE.lock().await;
    // ...
}
```

The library has 80+ internal functions. With State, all of them would need `&State` — even those that just call something else.

What is `LazyLock`?

`std::sync::LazyLock` — a value initialized **once**, on first access, then shared immutably. Thread-safe. Stable since Rust 1.80.

```
use std::sync::LazyLock;

// Evaluated once, on first access. Never again.
static CONFIG: LazyLock<String> = LazyLock::new(|| {
    std::env::var("MY_CONFIG").expect("MY_CONFIG must be set")
});

fn anywhere() {
    println!("{}", *CONFIG); // Just use it. No parameter needed.
}
```

Like `lazy_static!` but in std. No macro, no extra crate.

LazyLock: Benefits & Trade-offs

Benefits

- **Zero boilerplate for users** - no `AppState` struct to create
- **No state threading** - 80+ internal functions access storage directly
- **Env-only config** - switch DB by changing `.env`, no code changes
- **Fail-fast init** - `init().await?` panics on invalid env at startup

Trade-offs

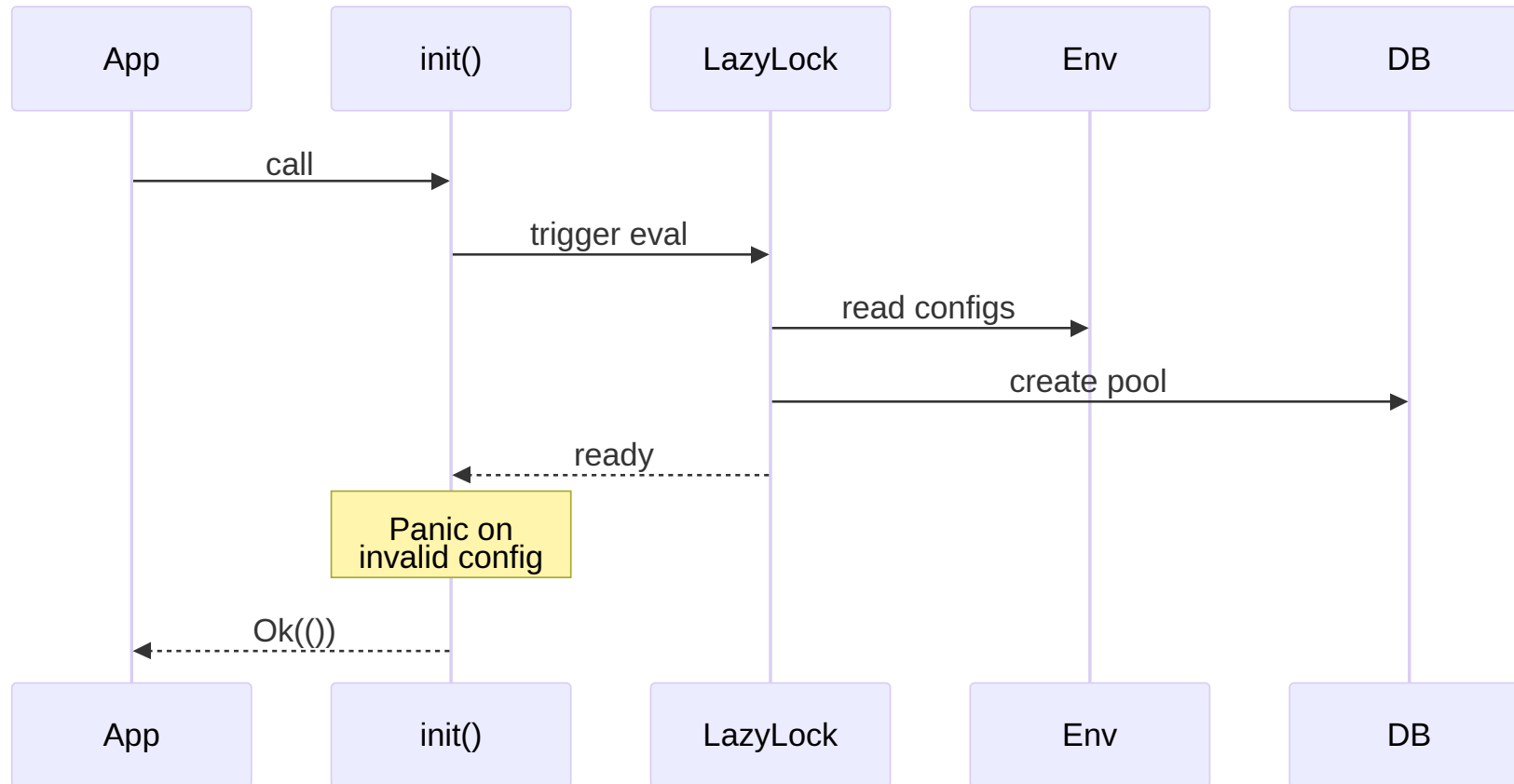
- Single instance per process (fine for auth library)
- Implicit dependencies (function signatures don't show DB access)
- Test isolation needs `#[serial]` (shared global state)

An intentional design trade-off: Rust purists would use `State`, but `LazyLock` keeps complexity inside the library.

LazyLock Initialization Flow

Fail-fast at Startup: Evaluates all configs/DB immediately.

No Axum State: Any handler can safely access `GENERIC_DATA_STORE` globally.



Rust Design Patterns Under the Hood

AuthUser: FromRequestParts Implementation

```
impl<B> FromRequestParts<B> for AuthUser
where
    B: Send + Sync,
{
    type Rejection = AuthRedirect;

    async fn from_request_parts(
        parts: &mut Parts, _: &B,
    ) -> Result<Self, Self::Rejection> {
        let method = parts.method.clone();

        // 1. Extract session cookie
        let session_cookie = cookies
            .get(SESSION_COOKIE_NAME.as_str())
            .ok_or_else(|| AuthRedirect::new(method.clone()))?;

        // 2. Validate session → get user
        let (session_user, csrf_token) =
            get_user_and_csrf_token_from_session(...)
                .await
                .map_err(|_| AuthRedirect::new(method.clone()))?;

        Ok(AuthUser::from(session_user))
    }
}
```

```
// AuthRedirect: redirect on GET, 401 otherwise
impl IntoResponse for AuthRedirect {
    fn into_response(self) -> Response {
        if self.method == Method::GET {
            Redirect::temporary(LOGIN_URL).into_response()
        } else {
            (StatusCode::UNAUTHORIZED,
             "Unauthorized").into_response()
        }
    }
}
```

```
// Option<AuthUser>: explicit OptionalFromRequestParts
impl<B> OptionalFromRequestParts<B> for AuthUser
where
    B: Send + Sync,
{
    type Rejection = AuthRedirect;

    async fn from_request_parts(...)
        -> Result<Option<Self>, Self::Rejection>
    {
        let result =
            <AuthUser as FromRequestParts<B>>
                ::from_request_parts(parts, state).await;
        Ok(result.ok()) // Err → None, no redirect
    }
}
```

Middleware: Why Not Just Use the Extractor?

`AuthUser` extractor:

- Always redirects on auth failure
- Always fetches user from DB
- Great for web pages that need user info

Problem: APIs should return 401 JSON, not redirect HTML. And sometimes you just need "is this user logged in?" without a DB query.

Middleware gives 2 axes of control:

Response type:

- `_redirect` → Web pages (GET redirects to login)
- `_401` → APIs (always returns 401)

Extension injected:

- `_user` → DB query, `Extension<AuthUser>`
- Non-`_user` → No DB query, `Extension<CsrfToken>` only

Variant	Unauthenticated	DB Query
<code>is_authenticated_401</code>	401	No
<code>is_authenticated_user_401</code>	401	Yes
<code>is_authenticated_redirect</code>	Redirect	No
<code>is_authenticated_user_redirect</code>	Redirect	Yes

Newtype Wrappers: Compile-Time Safety

```
pub struct Csrftoken(String);
pub struct UserId(String);
pub struct SessionId(String);
pub struct SessionCookie(String);
```

```
// Without newtypes - compiles but wrong!
fn check(session_id: &str, user_id: &str);
check(user_id, session_id); // Oops!
```

```
// With newtypes - compile error!
fn check(session_id: SessionId,
        user_id: UserId);
check(user_id, session_id); // Error!
```

```
impl UserId {
    pub fn new(id: String) -> Result<Self, SessionError> {
        if id.is_empty() {
            return Err(SessionError::Validation(
                "User ID cannot be empty".into()));
        }
        if id.len() > 255 {
            return Err(SessionError::Validation(
                "User ID too long".into()));
        }
        if !id.chars().all(|c| c.is_ascii_alphanumeric()
            || matches!(c, '-' | '_' | '.' | '@' | '+')) {
            return Err(SessionError::Validation(
                "Invalid characters".into()));
        }
        Ok(UserId(id))
    }
}
```

If you have a `UserId`, it's guaranteed valid. No re-validation needed.

Error Hierarchy with `thiserror`

```
#[derive(Error, Debug)]
pub enum CoordinationError {
    #[error("Unauthorized access")]
    Unauthorized,
    #[error("Resource not found: {resource_type} {resource_id}")]
    ResourceNotFound { resource_type: String, resource_id: String },

    // Wrap module-specific errors
    #[error("OAuth2 error: {0}")] OAuth2Error(OAuth2Error),
    #[error("Passkey error: {0}")] PasskeyError(PasskeyError),
    #[error("Session error: {0}")] SessionError(SessionError),
    #[error("User error: {0}")]   UserError(UserError),
}
```

Each module has its own error type. The coordination layer wraps them all.

From Impls with Auto-Logging

```
impl From<OAuth2Error>
  for CoordinationError
{
  fn from(err: OAuth2Error) -> Self {
    let error = Self::OAuth2Error(err);
    tracing::error!("{}", error);
    error // Auto-log on conversion!
  }
}
```

Every `?` that crosses a module boundary automatically logs:

```
// ? triggers From impl + logging
let token = exchange_code(code)
    .await?; // OAuth2Error -> logged
let user = get_user(id)
    .await?; // UserError -> logged
```

No manual `tracing::error!()` needed.

Security: Constant-Time Comparison

```
use subtle::ConstantTimeEq;

// CSRF token verification
if header_csrf_token
    .as_bytes()
    .ct_eq(session_csrf_token
        .as_str().as_bytes())
    .into()
{
    // Token matches
}
```

Why not just `==` ?

- Regular `==` short-circuits on first mismatch
- Attacker measures response time to guess bytes
- `ct_eq` always takes the same time

Used for CSRF tokens, session validation, and other security-sensitive comparisons.

CSRF Protection: Batteries Included

- **Auto-generated:** 32-byte random token created on every session creation
- **Auto-verified:** `X-CSRF-Token` header checked automatically in middleware & `AuthUser` extractor
- **Auto-delivered:** Token included in every authenticated response header
- **Dedicated endpoint:** `GET /o2p/user/csrf_token` → JSON `{ "csrf_token": "..."}`

Request	Result
GET / HEAD / OPTIONS	Skipped (safe methods)
POST + <code>X-CSRF-Token</code> header 	Auto-verified via <code>ct_eq</code> — mismatch → reject
POST + no header + form Content-Type	Passed to handler (manual check)
POST + no header + other Content-Type	Rejected (403)

CSRF: Usage Patterns

AJAX — fully automatic

```
// 1. Get token from response header
const csrf = response.headers.get('X-CSRF-Token');

// 2. Include in subsequent requests
fetch('/api/action', {
  method: 'POST',
  headers: { 'X-CSRF-Token': csrf },
  credentials: 'include',
  body: JSON.stringify(data),
});
// Middleware verifies automatically ✓
```

HTML Form — manual check in handler

```
<input type="hidden" name="csrf_token"
      value="{ csrf_token }">
```

```
async fn handler(
  user: AuthUser,
  Form(form): Form<MyForm>,
) -> impl IntoResponse {
  if !user.csrf_via_header_verified {
    if !form.csrf_token.as_bytes()
      .ct_eq(user.csrf_token.as_bytes()).into()
    {
      return StatusCode::FORBIDDEN.into_response();
    }
  }
  // ...
}
```


Atomic SQL: No Explicit Transactions

```
// "Delete user, but only if they're not the last admin"
// Done in a SINGLE SQL statement - no transaction needed

pub async fn delete_user_if_not_last_admin_postgres(
    pool: &Pool<Postgres>, id: UserId,
) -> Result<bool, UserError> {
    let result = sqlx::query(&format!(
        r#"DELETE FROM {table}
           WHERE id = $1
           AND (SELECT COUNT(*) FROM {table}
                WHERE is_admin = true) > 1"#
    ))
    .bind(id.as_str())
    .execute(pool).await?;

    Ok(result.rows_affected() > 0)
}
```

Business logic in SQL = atomic by default. No race conditions.

SQL Dialect Differences

PostgreSQL - one query

```
sqlx::query_as::<_, User>(&format!(  
    "INSERT INTO {table} (...)  
    VALUES ($1, $2, ...)  
    ON CONFLICT (id) DO UPDATE SET ...  
    RETURNING *"  
)).fetch_one(pool).await
```

SQLite - need two queries

```
sqlx::query(&format!(  
    "INSERT INTO {table} (...)  
    VALUES (?, ?, ...)  
    ON CONFLICT (id) DO UPDATE SET ..."  
)).execute(pool).await?;  
  
sqlx::query_as::<_, User>(&format!(  
    "SELECT * FROM {table} WHERE id = ?"  
)).fetch_one(pool).await
```

Supporting 3 databases means handling these quirks per-backend.

Integrating with Your App

Integrating Your App: 1:N (demo-todo)

```
CREATE TABLE todos (  
  id          SERIAL PRIMARY KEY,  
  user_id     TEXT NOT NULL,      -- FK to users.id  
  title       TEXT NOT NULL,  
  completed   BOOLEAN DEFAULT FALSE  
);  
CREATE INDEX idx_todos_user_id ON todos(user_id);
```

```
pub fn todos_router() -> Router<AppState> {  
  Router::new()  
    .route("/todos", get(list_todos).post(create_todo))  
    .route_layer(from_fn(is_authenticated_redirect))  
}
```

```
async fn create_todo(  
  State(state): State<AppState>,  
  user: AuthUser,  
  Form(form): Form<TodoForm>,  
) -> Result<Response, ...> {  
  db::create_todo(  
    &state.pool,  
    &user.id,      // —> saved as todos.user_id  
    &form.title,  
  ).await?;  
  Ok(Redirect::to("/").into_response())  
}
```

`AuthUser.id` is the only thing your app needs from the library — everything else is your own DB.

Integrating Your App: 1:1 (demo-profile)

```
CREATE TABLE user_profiles (  
  user_id      TEXT PRIMARY KEY,  -- PK = FK: enforces 1:1  
  display_name TEXT,  
  bio         TEXT,  
  avatar_url  TEXT  
);
```

```
pub fn profile_router() -> Router<AppState> {  
  Router::new()  
    .route("/profile", get(show_profile).post(update_profile))  
    .route_layer(from_fn(is_authenticated_redirect))  
}
```

```
async fn update_profile(  
  State(state): State<AppState>,  
  user: AuthUser,  
  Form(form): Form<ProfileForm>,  
) -> Result<Response, ...> {  
  db::upsert_profile(  
    &state.pool,  
    &UserProfile {  
      user_id: user.id.clone(),  
      bio: form.bio,  
      ..Default::default()  
    },  
  ).await?;  
  Ok(Redirect::to("/").into_response())  
}
```

`user_id TEXT PRIMARY KEY` enforces 1:1 at the DB level — no separate `id` column needed.

Live Coding: Putting It All Together

What We'll Build

Everything we just covered, live, from an empty directory: `cargo new` to a working login with a protected page — in real time.

1. `step-01` — `cargo new` + add dependencies
2. `step-02` — `.env` for SQLite (zero setup)
3. `step-03` — `init()` + merge router → **built-in login UI works**
4. `step-04` — protect a route with the `AuthUser` extractor
5. `step-05` — same route, but anonymous access allowed (`option<AuthUser>`)
6. `step-06` — swap the extractor for middleware — 401 vs redirect

Each step is a tagged git commit. If I fall behind, I jump straight to the tag instead of typing it out — same code either way.

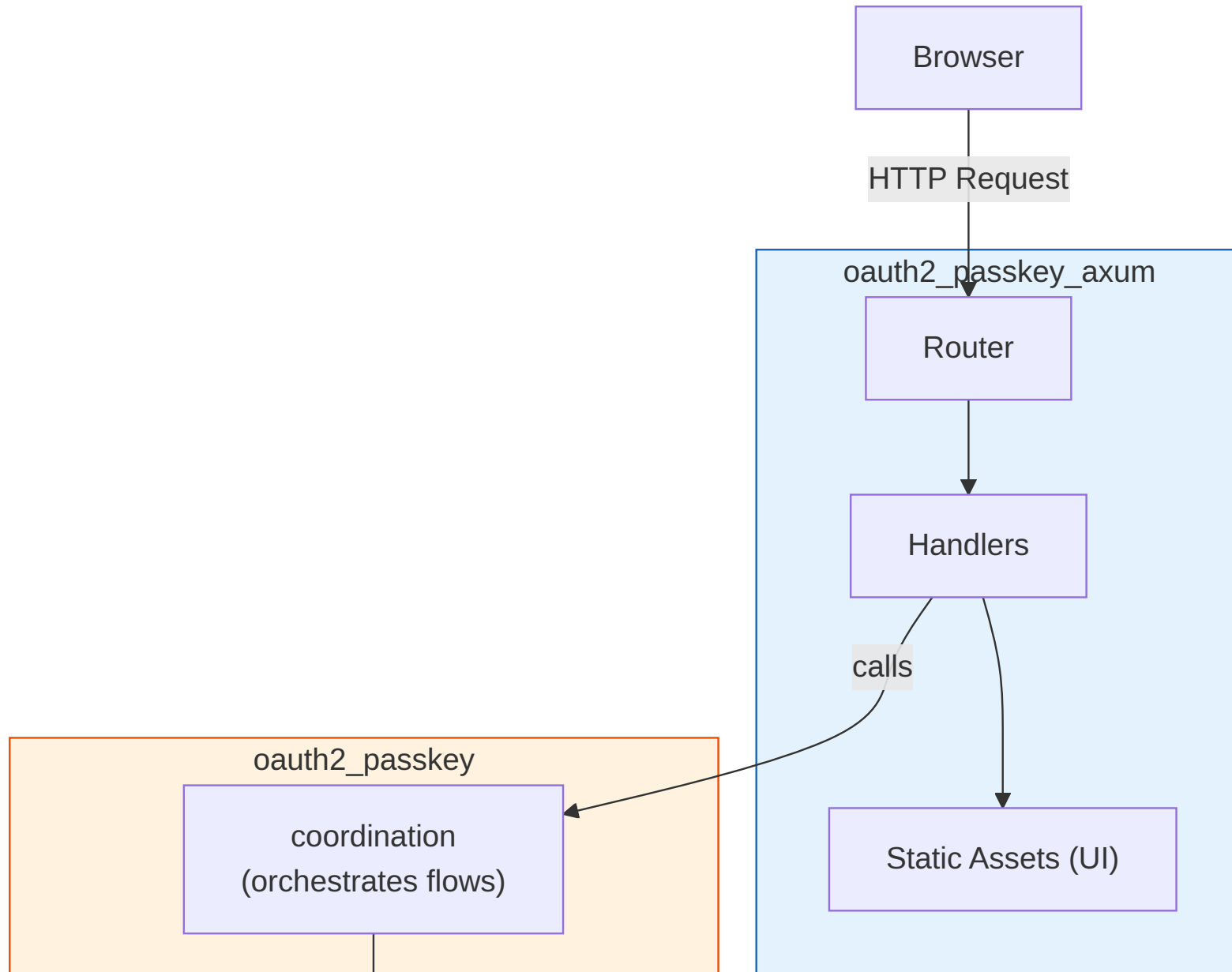
Live Coding Recap

Tag	What it adds
step-01	Empty Axum project + deps
step-02	<code>.env</code> — SQLite, no external DB
step-03	<code>init()</code> + merged router → working login UI
step-04	<code>AuthUser</code> extractor on a protected route
step-05	<code>Option<AuthUser></code> — same route, anonymous allowed
step-06	Middleware variant — 401 without a redirect

From an empty directory to session-backed auth with account management: **~6 commits**.

Wrap-up

Crate Structure



Future Plans

- **DPoP (Demonstration of Proof-of-Possession)**
 - Bind tokens to a cryptographic key
 - Prevent token theft/replay
- **Bearer Token support**
 - API authentication beyond session cookies
- **FedCM (Federated Credential Management)**
 - Browser-native identity UI (experimental, already partially implemented)
- **More OAuth2 providers**

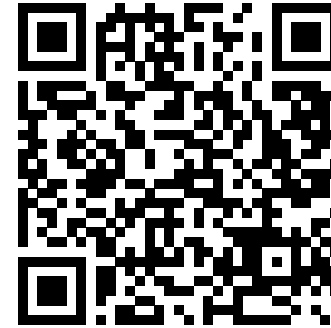
Summary

- **Easy:** Add passwordless auth to your Axum app in minutes — Built-in UI included
- **Secure:** Passkey (phishing-resistant) + OAuth2, with CSRF protection and secure session cookies
- **Flexible:** Switch between SQLite, PostgreSQL, MySQL, and Redis with a single `.env` change
- **Transparent:** LazyLock globals, newtypes, and `thiserror` hierarchies keep the internals inspectable

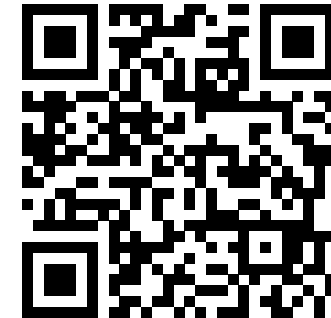
Thank You! / Questions?

About me:

- Kimitoshi Takahashi
- Self-employed, reskilling in Rust
- Let's start a startup together!



GitHub



Contact

Extra Slides

Testing Strategy

- Integration tests run against **every** backend: SQLite, PostgreSQL, MySQL
- Shared global storage (`LazyLock`) means tests can't run concurrently against the same backend
- `#[serial]` (from the `serial_test` crate) forces those tests to run one at a time
- Trade-off accepted deliberately — same reasoning as the LazyLock design itself

From/Into: Clean Layer Boundaries

```
// DB layer -> Session layer
impl From<DbUser> for SessionUser {
    fn from(db_user: DbUser) -> Self {
        Self {
            id: db_user.id,
            account: db_user.account,
            label: db_user.label,
            is_admin: db_user.is_admin,
            ..
        }
    }
}
```

```
// Session -> Axum layer
impl From<SessionUser> for AuthUser {
    fn from(su: SessionUser) -> Self {
        AuthUser {
            id: su.id,
            // ... copy fields
            csrf_token: String::new(),
            session_id: String::new(),
            // ^ Axum-specific fields
        }
    }
}
```

Each layer has its own type. `From` impls make conversion seamless with `into()`.